Toward Multi-Robot Construction Planning with Dec-POMDPs

Preston Culbertson and Patrick Slade

***Abstract*— In this work, we aim to allow a team of robotic agents to develop decentralized policies for fabricating structures under uncertainty. The multi-robot construction problem is posed as a Decentralized Partially Observable Markov Decision Process (Dec-POMDP), and we formulate transition, observation, and reward models for the multi-agent problem. We first study the problem of single-agent construction in a fully observable setting to explore how agents optimize their construction activities, and to observe "scaffolding," the placement of unnecessary blocks which allow the agent to reach higher areas of the structure. We then study a centralized, fully observable multi-agent construction problem, as well as a single agent performing construction tasks in a partially observable domain. A decentralized framework is then posed for how this work could be extended to account for a lack of central information passed between agents. This work represents our initial steps toward enabling decentralized multi-robot construction with Dec-POMDPs.**

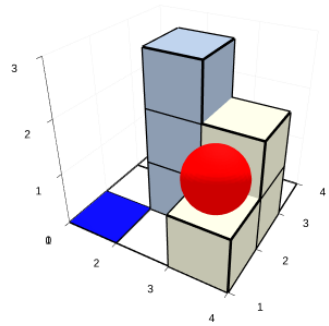

Fig. 1: Assembly of block tower with scaffolding

I. Introduction

A. Motivation

While there has been a recent interest in multi-robot construction of structures, there exist few decentralized construction methods that can operate in unstructured, noisy environments. Further, the decentralized construction methods proposed to date do not attempt to optimize their construction plans according to some cost function, but often can only guarantee collision avoidance and the correctness of the final structure.

We seek a decentralized, robust method for multi-robot construction. To this end, we propose to formulate multi-robot construction initially as a multi-agent Markov Decision Process (MDP), extend this to a Partially Observable MDP (POMDP), with the ultimate aim of adding a decentralized aspect to formulate a Dec-POMDP. A Dec-POMDP is an extension of a POMDP which models multiple agents acting in a partially observable, stochastic domain, with rewards that are based on the behavior of the entire group. By modeling our problem as a Dec-POMDP, we can encode both the stochastic dynamics and a reward model, while its solution yields a decentralized construction plan. The worst-case complexity of computing a solution to a Dec-POMDP has been shown to be NEXP-Complete, but there exist approximate methods which are computationally tractable and yield near-optimal solutions.

B. Related Work

While multi-robot construction has been demonstrated in literature, a generalized approach to construction planning for robotic teams remains largely unstudied. In [1], the authors used teams of quadrotors to construct cubic structures using self-assembling beams. The agents' policies were modeled as Finite State Automotons (FSAs) which switched between different desired behaviors (e.g. "pick up part," "place part") to construct the desired structure. In [2], the authors use a team of ground-based robots to assemble IKEA furniture. A desired model of the furniture is sequenced into higher-level symbolic specifications, and an open-loop plan is formulated offline for its assembly. Work in [3] offers a general decentralized algorithm for performing 3D robotic assembly with different types of parts with with probable constant bounds of deviation and adaptability to avoid failure. Biologically inspired work in [4] used a multi-agent construction system that generates low-level rules for independent robots with local-sensing that guarantee production of a structure.

C. Contribution

In this project, we develop a toy, gridworld-based model of multi-robot construction, and formalize it as a Dec-POMDP. We then study numerous simplifications of the problem, including single- and multi-agent construction tasks in fully and partially observable domains. These problems are solved via online, sampling-based planners, and exhibit emergent behavior such as "scaffolding," placing unnecessary blocks to reach higher areas of the state space.

II. Background

A. MDPs, POMDPs, Dec-POMDPs

An MDP is a mathematical framework for a sequential decision-making process in which an agent moves stochasti-

The authors are with the Department of Mechanical Engineering, Stanford University, Stanford, CA 94305 USA (e-mail: pculbert,patslade@stanford.edu).

cally between states over time, accruing various rewards for entering certain states. In this framework, agents may affect their trajectory and rewards by taking actions at each time step. An MDP is defined by the tuple $(\mathcal{S}, \mathcal{A}, T, R, \gamma)$, where:

- $\mathcal{S}$ is the set of states the system may reach,
- $\mathcal{A}$ is the set of actions the agent may take,
- $T(s' \mid s, a)$ is the probability of transitioning to state s' by taking action a at state s,
- $R(s, a)$ is the reward (or cost) of taking action a at state s, and
- $\gamma \in [0, 1]$ is the discount factor for future rewards.

Solving an MDP involves developing a policy, $\pi(s) : \mathcal{S} \rightarrow \mathcal{A}$, which maps each state to an optimal action that will accrue the most rewards in expectation over some planning horizon.

For infinite-horizon, discounted MDPs, the optimal policy has been shown to satisfy the Bellman equations:

$$\pi^*(s) = \underset{a}{\operatorname{argmax}}\Big(R(s, a) + \gamma \sum_{s'} T(s' \mid s, a) U^*(s')\Big), \quad (1)$$

where

$$U^*(s) = \max_{a}\Big(R(s, a) + \gamma \sum_{s'} T(s' \mid s, a) U^*(s')\Big). \quad (2)$$

For small, discrete state and action spaces, the Bellman equations may be solved explicitly with dynamic programming (DP). However, for problems with large or continuous state and action spaces, only approximate solutions may be found using approximate dynamic programming (ADP) [5]. Below, we will discuss Monte Carlo Tree Search (MCTS), the online ADP algorithm applied in this work.

A POMDP is an extension of an MDP, where the agent cannot directly observe its true state, only taking observations which are stochastically dependent on this state. A POMDP thus contains a model of the probability of seeing a certain observation at a specific state. The agent then forms a belief state, b which encodes the probability of being in each state s. The agent updates its belief at each step, depending on its previous action, the reward received, and the observation it took. Since the agent may hold any combination of beliefs about its location in the state space, the belief space $\mathcal{B}$ is continuous, making POMDPs computationally intensive to solve.

A Dec-POMDP requires changes to the initial POMDP definitions to handle multiple agents. A Dec-POMDP is defined by the tuple $(I, \mathcal{S}, \{\mathcal{A}_i\}, T, R, \{\Omega_i\}, O)$, where:

- I is the set of agents,
- $\mathcal{S}$ is the set of states the system may reach,
- $\mathcal{A}_i$ is the set of actions agent i may take,
- $T(s' \mid s, a)$ is the probability of transitioning to state s' by taking action a at state s,
- $R(s, a)$ is the reward (or cost) of taking action a at state s,
- Ω_i is the set of observations for agent i
- $O(o \mid s, a)$ is probability of seeing observation o while in state s and taking action a.

B. Monte-Carlo Tree Search (MCTS)

MCTS is a sampling-based online approach for approximately solving MDPs. MCTS requires a generative model G that can draw a random state and reward $(s', r) \sim G(s, a)$. The algorithm performs a forward search through the state space, using G to draw prospective trajectories and rewards. In MCTS, a tree is created with alternating states and actions, with a roll-out performed to select a policy [6]. In general, MCTS involves four stages:

- *Selection*: Traverse the tree to a leaf node of choice, storing the accrued rewards and number of times each node is visited.
- *Expansion*: If at a leaf node, take a new action, and add the resulting new state to the tree.
- *Rollout*: At each leaf node, perform a "roll out" simulation according to some pre-determined (or random) policy to estimate this state's value.
- *Propagation*: Propagate the new state's estimated values along the taken path.

C. Partially Observable Monte-Carlo Planning (POMCP)

POMCP [7], developed by Silver et al., is an application of MCTS to planning in partially observable domains. Since the states of a POMDP are not fully observable, a tree is formed over belief states for planning. However, unlike in an MDP, the tree must branch twice, once for actions, and again for the possible observations that may be received. POMCP thus constructs a belief tree according to the UCT heuristic, using a generative model G' which returns a state-observation-reward tuple, $(s', o, r) \sim G'(s, a)$, and uses a particle filter to maintain and update its belief.

As we will discuss later, the construction POMDPs considered in this paper suffer heavily from the "curse of dimensionality," resulting in massive state spaces with sparse transition distributions. These properties motivate our use of sampling-based online solvers, compared to offline (PO)MDP solution methods such as (Point-Based) Value Iteration which must construct policies over a large number of states which are likely unreachable or irrelevant.

III. Problem formulation

A. Construction MDP

Cooperative manipulation for box pushing in a setting similar to construction planning has been framed as a Dec-POMDP [8]. We formalize the construction planning framework in a similar Dec-POMDP setup. A given number of robot agents, n, are located in a grid world setup of size G. Some locations of the grid are designated as depots, $\mathbb{D}_i$, where the agents can access blocks to fabricate the structure. Further, each space in the grid has a desired number of blocks, $H_i \in \mathbb{N}$, where $\mathbb{N}$ denotes the natural numbers. We define the "blueprint" B of the construction problem to be the set $\{H_1, \ldots, H_G\}$ of desired bricks for each square, where $H_i = -1$ if square i is a depot.

We thus can construct the state space, $S = S_e \times_i S_i$, where S_e denotes the environment state, a vector of $h_j \in$

$\{-1, 0, \ldots, H\}$, for $j \in \{1, \ldots, G\}$, where $H = \max_j H_j$, and $S_i \in \{1, \ldots, G\} \times \{0, 1\}$, which denotes the grid tile occupied by robot i, and whether it is currently carrying a block. Thus, we can write $|S| \leq (H+2)^G G^{2n}$. Figure 2 shows a overhead view of an example gridworld, with $n = 2$ robots. Figure 3 shows a side view of the desired structure.

Each agent can act to move, place a block, or pick up a block in the north, south, east, and west directions. This results in an individual action space with cardinality $|\mathcal{A}_i| = 12$. In the multi-agent case, the joint action space is defined by $\mathcal{A} = \times_{i=1}^{n} \mathcal{A}_i$, with $|\mathcal{A}| = 12^n$.

We now define the reward model for this problem. The construction team receives a massive joint reward $R = 10,000$ for completing the structure (i.e. the environment state $S_e = B$, the construction blueprint. Further, the team receives a moderate reward $R = 100$ for placing a block in the structure, and a large negative reward $R = -300$ for removing a block from the structure. This reward structure allows the construction team to "spread" the large reward for completing the structure back to intermediate states, which allows for easier solving via Monte-Carlo methods, while also incentivizing forward progress (i.e. preventing the agents from "reward hacking" via placing and removing the same block *ad infinitum*).

Finally, we can shape the rewards by giving a very small reward $R = 5$ for placing any block if the structure is unfinished ($S_{ei} \leq B_i, \forall i$), and for removing any block otherwise. This again incentivizes forward progress, and aids in solutions via sampling-based planners. This shaping was only used when building tall structures (i.e. towers of height 3 or more) which required extensive scaffolding. Any other action yields a reward $R = -1$.

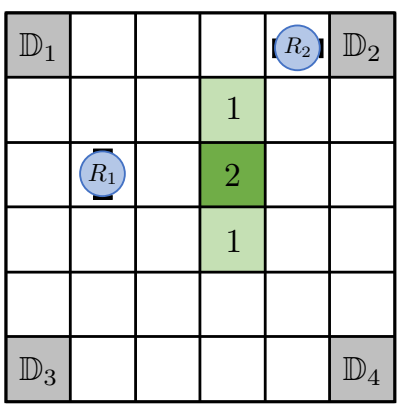

Fig. 2: Schematic of Observation for Agent 2

Extensions of this initial framework would allow for more freedom in expressing the relation between parts such as order of assembly, different types of components, or collaborative effort needed to move them. A grid-world framework is quite restrictive on the type of structures since most consist of non-square parts, and contain voids. Incorporating this would allow for more automatic synthesis of a construction plan from general design documents.

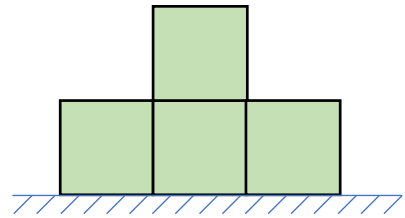

Fig. 3: Side View of Desired Structure

B. Partial Observability

In construction tasks, there often exist occlusions in the structure which prevent coworkers from observing each other's actions, and the progress of the entire structure. Thus, we wish to capture this phenomena via a partially-observable model. In this work, we model partial observations by only allowing each agent to measure the environment state of its own grid tile, and those adjacent to it (along the cardinal and diagonal directions), which results in 9 measurements total. The agents receive a special measurement of `null` when the adjacent tile is a wall, and `R_i` when they are adjacent to robot i. Figure 4 shows a representative observation for agent R_2 in the example gridworld shown in Figure 2.

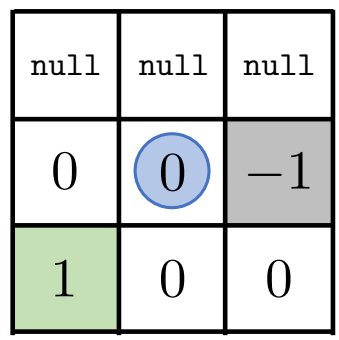

Fig. 4: Schematic of Gridworld Construction Problem

We note that this observation model, while partial, is deterministic and somewhat trivial for the single-agent case. If, as in this work, the agent models its belief over the underlying states as a particle filter, then this estimate will remain spread out in the initially empty world, but quickly converge once the agent localizes itself (e.g. observes a corner or a uniquely placed depot). Afterwards, since the environment state can only change through the actions of the agent, the particles will remain entirely on the true world state, meaning this problem is nearly fully observable.

We chose to study this simple observation model due to the significant complexity it introduces for decentralized teams of agents, since the environment state is determined by the joint actions of the construction team. In future work, we hope to study how agents can propagate their belief for the hidden environment states, which may be changing due their teammates' actions. Complexity may be easily added to this model by allowing for noisy measurements, or by modeling occlusions via "laser beams" which stop at walls or occlusions as in the *LaserTag* problem given in [9].

IV. Approach

In this section, we present our approaches to solving three construction (PO)MDPs: a single agent acting in a fully observable domain, multiple agents acting in a fully observable domain, and finally a single agent in a partially observable domain. Although, in the long term, we are interested in decentralized policies for the partially observable problem (which motivates a Dec-POMDP approach), we chose to study these simpler problems to gain insights about our model. We hope to leverage these insights in our future work toward a Dec-POMDP-based construction model.

All code for this project was written in Julia, and all solvers (MCTS, POMCP) and particle filters used in our implementations were sourced from `POMDPs.jl` [10], a package which provides a large number of tools for modelling and solving (PO)MDPs.

A. Single-agent MDP

Initially, an MDP framework was implemented with a single agent following the previously described transition function and assuming full observability. This demonstrated initial feasibility for using an MDP framework to plan at the low-level of moving, picking up, and placing blocks. It also investigated what kind of solutions would emerge from the (PO)MDP framework, such as adding scaffolding to build a tower multiple blocks high and then removing it to achieve the desired structure. The agent's policy was generated at each step with MCTS, which used a random rollout policy to estimate the value of the leaf nodes.

B. Centralized multi-agent

An extension of this MDP framework was then made to test solutions with multiple agents in a fully observable, centralized setting. The same transition dynamics were used for each agent individually but if any agents attempted to perform the same action on any block, other than receiving a block from the same depot, a large penalty for a collision was given as a reward to encourage the agents to perform different tasks and prevent them from interfering with each other.

The multi-agent policies were again generated by MCTS, which searched over the joint state-action space. Since the action space cardinality is exponential in the number of agents, sampling-based methods would likely produce low-quality centralized policies. However, for our toy problems, with two agents, we found MCTS yielded good results.

C. Partially observable single-agent MDP

Finally, we studied the case of a single agent in a partially observable domain, with local observations described in Section III-B. In this case, we placed the agent on a 3×3 gridworld, with a blueprint containing a desired 2-block tower on the middle square. The agent used a sequential importance resampling particle filter to maintain its belief about the problem state. The agent's policy was generated using POMCP with a custom heuristic policy that was rolled out to estimate the value of the leaf nodes. Algorithm 1 shows the pseudocode for the heuristic.

Algorithm 1 Heuristic policy for Construction POMDP

```
if h_i ≤ B_i, ∀i then
    if agent holding block then
        place at closest needed tile
    else
        drop block at depot
    end if
else
    if agent holding block then
        drop block at depot
    else
        pick up closest scaffolding block
    end if
end if
```

This heuristic policy represents a simple state-machine-based policy for gridworld construction that is similar to many decentralized construction policies which are found in literature. If the structure is unfinished, i.e. there exists a grid tile with fewer blocks than required by the blueprint, then the heuristic returns a policy which either retrieves a block, or attempts to place a block at the closest "unfinished" grid tile. If the structure is finished, this policy operates in reverse; the agent greedily picks up scaffolding blocks, and drops them back into the depot. We note that this policy can only greedily complete structures of height 1, since it is unable to perform scaffolding or to navigate the movement constraints induced by a vertical structure.

V. Simulation and Results

A. Single-agent MDP

Simulations were run for a 5×5 gridworld where the agent built an 'S' shape with depots at each corner shown in Fig. 5. Due to the simple structure of placing blocks only to a height of 1 the agent was able to accomplish this efficiently, utilizing all depots available.

A simple 3 block high tower was simulated on a 3 by 3 grid with a single depot. Although the structure was infeasible to build without adding additional blocks, based on our transition model, with some reward shaping to promote placing additional blocks when no blocks could be directly added to the desired structure the agent saw emergent behavior of forming scaffolding allowing it to place higher blocks than could be reached from on the desired structure. Similar reward shaping was applied to encourage the disassembly of the scaffolding blocks to leave only the desired structure. The agent was able to determine the order in which the scaffolding should be removed without causing any blocks to become unreachable and require an additional block to be placed, thus removing the higher blocks first.

B. Centralized multi-agent

A simulation with 2 agents in a 3×3 grid built a larger structure similar to the 3 block tower but with scaffolding

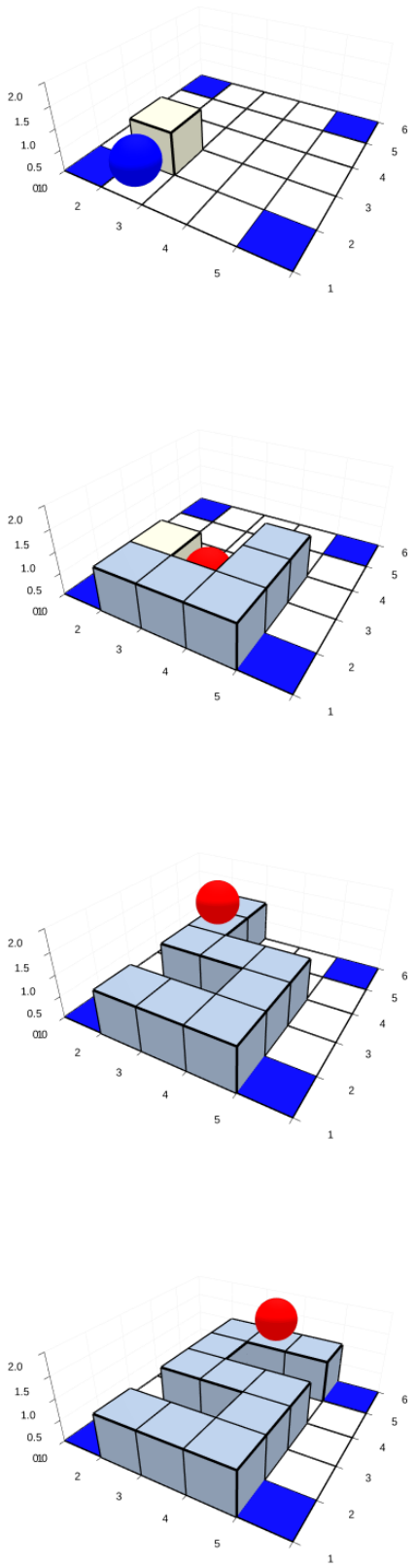

Fig. 5: Assembly of 'S' shape

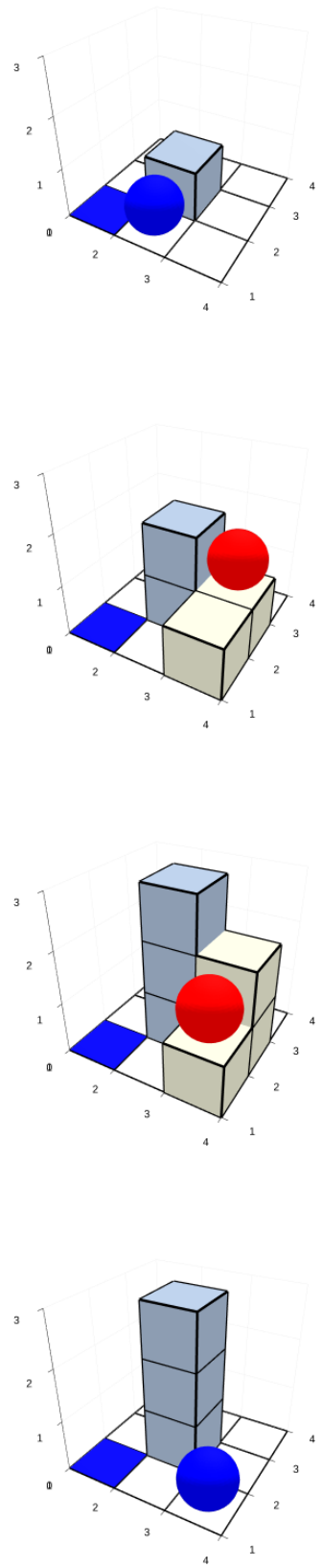

Fig. 6: Assembly of 3 block tower with scaffolding

built into the structure on both sides. The agents were able to succeed in building the structure, but due to the small size of the gridworld the agents, especially towards the end of construction when there was only a single block left, one or more agents would not perform any actions to prevent any collisions and perform very cautiously. Further tuning of the reward structure might encourage more collaborative behavior than just trying to operate independently without causing any accidents to occur between agents.

C. Single-agent POMDP

We tested the single agent POMDP solution on both the 2-block tower and the Stanford 'S' construction tasks. As discussed previously, since the partial observability is deterministic in our problem, the particle filter converged within the first step for both tasks. While the dynamics considered here were deterministic, the particle filter would become necessary in case the agent had stochastic transitions or observations, to model the belief distributions possible under those dynamics.

The agent was able to successfully construct both the 2-tower and the 'S', and even outperformed the MDP solution in the 'S' construction task. This is likely due to the approximate nature of both solutions; we aim to perform more simulations from various initial states to better evaluate the relative performance of the algorithms.

However, even with a heuristic rollout policy, the POMCP policy was unable to construct the 3-block tower. We believe this is due to the sparse rewards present in the problem. Even with the shaped rewards, the agent would often get 'stuck' in states which were far from adding the next block to the structure, and take random actions. We aim to eliminate this problem by either incorporating a heuristic policy which is able to place scaffolding, by selecting better hyperparameters for POMCP, or by considering another algorithm which can better handle the sparse reward structure of this problem.

D. Summary of Results

Table I gives a summary of the simulation results for all scenarios. We note that the fully observable case is a lower bound for the performance of the partially observable case, which is as expected. Further, while the fully observable agent exhibited intricate scaffolding behavior, and could fabricate a tower of 3 blocks, the agent with partial observations was unable to do so. We believe this could be due to hyperparameter selection or particle filtering issues. Further, it should be noted that the MDP agent uses a random policy to estimate the value of its leaf nodes, compared to the handcrafted poicy used by the agent with partial observations.

In addition, we note that the multi-agent policy exhibited inferior performance to the single-agent cases on both construction tasks. There are a number of reasons this is possible. Most importantly, since we are considering very small state spaces (i.e. 3×3 and 5×5 gridworlds) for these construction tasks, both agents must operate in close proximity, and must take care to not collide or attempt to place the same block.

Problem	2-Tower	3-Tower	Stanford 'S'
Single-Agent MDP	68.8 *(10)*	13.7 *(32)*	55.2 *(50)*
Multi-Agent MDP	23.3 *(87)*	*N / A*	48.5 *(76)*
Single-Agent POMDP	6.85 *(13)*	*N / A*	55.3 *(47)*

TABLE I: Summary of Simulation Results for Construction (PO)MDPs [1]

These issues are compounded by the sampling-based planners we used for our (PO)MDP solution. To wit, the centralized planner must sample the entire joint action space (with $|\mathcal{A}| = 144$ for the 2-agent case), which results in a large number of suboptimal actions for an individual agent. Additionally, if the planner samples an action which causes the agents to collide (which is very likely on the small gridworlds studied), then it may choose to avoid branches of the tree which may have been optimal due to this bad sample. In all, we can confirm that the joint action space suffers heavily from the "curse of dimensionality," which further motivates our desire to develop decentralized policies which can avoid these issues and leverage the parallelism of multi-agent teams.

VI. Conclusions and Future Work

While existing approaches to robotic construction seek good heuristics which can guarantee the completion of a desired structure, we instead sought to develop methods which allow a team of agents to optimize their construction plan in a decentralized fashion. In this work, we defined a toy construction problem for a team of robotic agents as a (PO)MDP, and generated good approximate solutions via online Monte-Carlo methods.

We studied three cases, including a single agent building structures in both fully and partially observable domains, as well as a centralized planner for a team of two agents in a fully observable domain. In all cases, we used a MCTS-based planner to generate good approximate solutions, and we noted good emergent behavior such as "scaffolding," where agents strategically add extra blocks to the structure so they can access higher portions of the building to complete the structure.

In our future work, we hope to expand this framework to more complex partially observable domains (such as the *LaserTag*-based observation model we discussed earlier), and to develop decentralized algorithms for planning in the construction domain. Decentralized planning in a partially observable environment will prove especially challenging, since agents must have a transition model for the environment that captures the (not directly observable) behavior of their teammates. This can be approached in a number of ways. The agents may either model the environment as stochastic (i.e. desired blocks can appear randomly), and thus their POMDP solutions may encourage them to take

[1] Table entries are given by: Discounted Reward *(# of steps)*

information-gathering actions to maintain an accurate belief about the environment state.

A Dec-POMDP requires multiple agents to operate under uncertainty on the basis of different observations. A joint policy is a set of policies, one for each agent. A joint policy that maximizes expected utility is desired. Similar to other MDP formulations, dynamic programming can be used to generate exact solutions by solving for a set of T-step policy trees [11]. A heuristic search method such as multiagent A^* can be used to leverage an upper bound when choosing joint policies to find a solution [12]. Policy iteration can be applied to finite-state-controllers to develop optimal policies [13]. Approximate methods such as memory-bounded dynamic programming attempt to limit the number of trees retained to a fixed number to speed up the solution [14]. Joint equilibrium search simplifies the problem by iteratively holding all policies but one fixed and then updating just that policy. Several of these methods would be possible methods to explore in the future.

Further, since we used a simple, but effective, heuristic policy to estimate the value of the leaf nodes in the MCTS, agents could model the behavior of their teammates as a stochastic policy which chooses between the heuristic action and a random action with some probability. This would allow the agents to successfully propagate their belief about the environment state, while also capturing the ways in which individual agents may depart from the heuristic policy due to optimizations from the MCTS.

In all, these studies represent our preliminary steps toward forming a Dec-POMDP-based approach to multi-robot construction problems. Our model of the construction problem as a POMDP is one of the first attempts to allow a team of agents to optimize their construction plan, and our partially observable model allows us to encode sophisticated effects such as occlusions and stochasticity into our construction model. We aim to leverage the techniques and insights developed in this work to continue our studies of multi-agent construction, and to better understand how to formulate good decentralized policies for teams of agents collaborating in partially observable environments.

VII. Acknowledgements

The authors would like to thank Zachary Sunberg for helpful discussion about POMDPs.jl functions.

References

[1] Quentin Lindsey, Daniel Mellinger, and Vijay Kumar. Construction with quadrotor teams. *Autonomous Robots*, 33(3):323–336, 2012.

[2] Ross A Knepper, Todd Layton, John Romanishin, and Daniela Rus. Ikeabot: An autonomous multi-robot coordinated furniture assembly system. In *Robotics and Automation (ICRA), 2013 IEEE International Conference on*, pages 855–862. IEEE, 2013.

[3] Seung-kook Yun and Daniela Rus. Adaptive coordinating construction of truss structures using distributed equal-mass partitioning. *IEEE Transactions on Robotics*, 30(1):188–202, 2014.

[4] Justin Werfel, Kirstin Petersen, and Radhika Nagpal. Designing collective behavior in a termite-inspired robot construction team. *Science*, 343(6172):754–758, 2014.

[5] Warren B. Powell. *Approximate Dynamic Programming: Solving the Curses of Dimensionality*. Wiley, 2nd edition, 2011.

[6] Cameron B Browne, Edward Powley, Daniel Whitehouse, Simon M Lucas, Peter I Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. A survey of Monte Carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1):1–43, 2012.

[7] David Silver and Joel Veness. Monte-carlo planning in large pomdps. In *Advances in neural information processing systems*, pages 2164–2172, 2010.

[8] Christopher Amato, George Konidaris, Gabriel Cruz, Christopher A Maynor, Jonathan P How, and Leslie P Kaelbling. Planning for decentralized control of multiple robots under uncertainty. In *Robotics and Automation (ICRA), 2015 IEEE International Conference on*, pages 1241–1248. IEEE, 2015.

[9] Adhiraj Somani, Nan Ye, David Hsu, and Wee Sun Lee. Despot: Online pomdp planning with regularization. In *Advances in neural information processing systems*, pages 1772–1780, 2013.

[10] Maxim Egorov, Zachary N. Sunberg, Edward Balaban, Tim A. Wheeler, Jayesh K. Gupta, and Mykel J. Kochenderfer. POMDPs.jl: A framework for sequential decision making under uncertainty. *Journal of Machine Learning Research*, 18(26):1–5, 2017.

[11] Eric A Hansen, Daniel S Bernstein, and Shlomo Zilberstein. Dynamic programming for partially observable stochastic games. In *AAAI*, volume 4, pages 709–715, 2004.

[12] Daniel Szer, Francois Charpillet, and Shlomo Zilberstein. MAA*: A heuristic search algorithm for solving decentralized POMDPs. In *21st Conference on Uncertainty in Artificial Intelligence-UAI'2005*, 2005.

[13] Daniel S Bernstein, Christopher Amato, Eric A Hansen, and Shlomo Zilberstein. Policy iteration for decentralized control of Markov decision processes. *Journal of Artificial Intelligence Research*, 34(1):89, 2009.

[14] Sven Seuken and Shlomo Zilberstein. Memory-bounded dynamic programming for dec-pomdps. In *IJCAI*, pages 2009–2015, 2007.